

In []: Final Project - Python - Data Visualization

```
In [1]: # =====
# AI Job Automation Risk Analysis - Python Visualization
# =====

# 1. Load required libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Optional library for feature importance
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, r2_score
```

```
In [7]: # 2. Load dataset

file_path = r"C:\Users\rable\OneDrive\Desktop\Data Visualization Coursework\AI JOB

df = pd.read_excel(file_path)

# Display first 5 rows
df.head()
```

```
Out[7]:
```

	job_id	job_role	industry	country	year	automation_risk_percent	ai_replaceme
0	0	Data Analyst	Technology	Canada	2021	26.22	
1	1	Accountant	Finance	Brazil	2020	52.08	
2	2	Teacher	Technology	USA	2020	31.30	
3	3	Customer Support Rep	Technology	Brazil	2021	56.92	
4	4	Teacher	Manufacturing	Japan	2024	14.55	

◀ ◯ ▶

```
In [8]: # 3. Understand dataset

print("Dataset Shape:", df.shape)
print("\nColumn Names:")
print(df.columns)

print("\nData Types:")
print(df.dtypes)
```

Dataset Shape: (15000, 20)

Column Names:

```
Index(['job_id', 'job_role', 'industry', 'country', 'year',  
      'automation_risk_percent', 'ai_replacement_score', 'skill_gap_index',  
      'salary_before_usd', 'salary_after_usd', 'salary_change_percent',  
      'skill_demand_growth_percent', 'remote_feasibility_score',  
      'ai_adoption_level', 'education_requirement_level',  
      'automation_risk_category', 'skill_transition_pressure',  
      'wage_volatility_index', 'reskilling_urgency_score',  
      'ai_disruption_intensity'],  
      dtype='object')
```

Data Types:

job_id	int64
job_role	object
industry	object
country	object
year	int64
automation_risk_percent	float64
ai_replacement_score	float64
skill_gap_index	float64
salary_before_usd	float64
salary_after_usd	float64
salary_change_percent	float64
skill_demand_growth_percent	float64
remote_feasibility_score	float64
ai_adoption_level	float64
education_requirement_level	int64
automation_risk_category	object
skill_transition_pressure	float64
wage_volatility_index	float64
reskilling_urgency_score	float64
ai_disruption_intensity	float64
dtype: object	

In [11]: *# 4. Basic statistics before missing value check*

```
df.describe()
```

Out[11]:

	job_id	year	automation_risk_percent	ai_replacement_score	skill_gap
count	15000.000000	15000.000000	15000.000000	15000.000000	15000.000000
mean	7499.500000	2022.997200	46.176347	46.155907	50.000000
std	4330.271354	1.999365	21.663635	22.351347	28.867513
min	0.000000	2020.000000	5.000000	4.010000	0.000000
25%	3749.750000	2021.000000	28.790000	28.357500	25.000000
50%	7499.500000	2023.000000	46.235000	45.675000	49.999999
75%	11249.250000	2025.000000	63.602500	62.712500	75.000000
max	14999.000000	2026.000000	94.980000	113.070000	99.999999



In [13]:

```
# 5. Mean, Median, Mode, Quartiles for important numeric columns
```

```
important_numeric_cols = [  
    "automation_risk_percent",  
    "ai_replacement_score",  
    "skill_gap_index",  
    "salary_before_usd",  
    "salary_after_usd",  
    "reskilling_urgency_score"  
]  
  
summary_stats = pd.DataFrame({  
    "Mean": df[important_numeric_cols].mean(),  
    "Median": df[important_numeric_cols].median(),  
    "Mode": df[important_numeric_cols].mode().iloc[0],  
    "Q1": df[important_numeric_cols].quantile(0.25),  
    "Q2": df[important_numeric_cols].quantile(0.50),  
    "Q3": df[important_numeric_cols].quantile(0.75),  
    "Min": df[important_numeric_cols].min(),  
    "Max": df[important_numeric_cols].max(),  
    "Std Dev": df[important_numeric_cols].std()  
})  
  
summary_stats
```

Out[13]:

	Mean	Median	Mode	Q1	Q2	
automation_risk_percent	46.176347	46.235	5.000	28.79000	46.235	6
ai_replacement_score	46.155907	45.675	33.690	28.35750	45.675	6
skill_gap_index	50.003708	49.930	4.050	25.17000	49.930	7
salary_before_usd	89771.375196	89533.050	41713.200	60127.22500	89533.050	11982
salary_after_usd	89870.633937	88787.330	39290.690	59223.45750	88787.330	11843
reskilling_urgency_score	35.867584	35.871	25.473	26.98225	35.871	4

In [15]: *# 6. Check missing values*

```
missing_values = df.isnull().sum()
missing_values[missing_values > 0]
```

Out[15]: Series([], dtype: int64)

In [17]: *# 7. Check duplicate records*

```
duplicate_count = df.duplicated().sum()
print("Number of duplicate records:", duplicate_count)
```

Number of duplicate records: 0

In [19]: *# 8. Handle missing values*

```
df_clean = df.copy()

# Fill numeric missing values with median
numeric_cols = df_clean.select_dtypes(include=["int64", "float64"]).columns

for col in numeric_cols:
    df_clean[col] = df_clean[col].fillna(df_clean[col].median())

# Fill categorical missing values with mode
categorical_cols = df_clean.select_dtypes(include=["object"]).columns

for col in categorical_cols:
    df_clean[col] = df_clean[col].fillna(df_clean[col].mode()[0])

# Remove duplicate records
df_clean = df_clean.drop_duplicates()

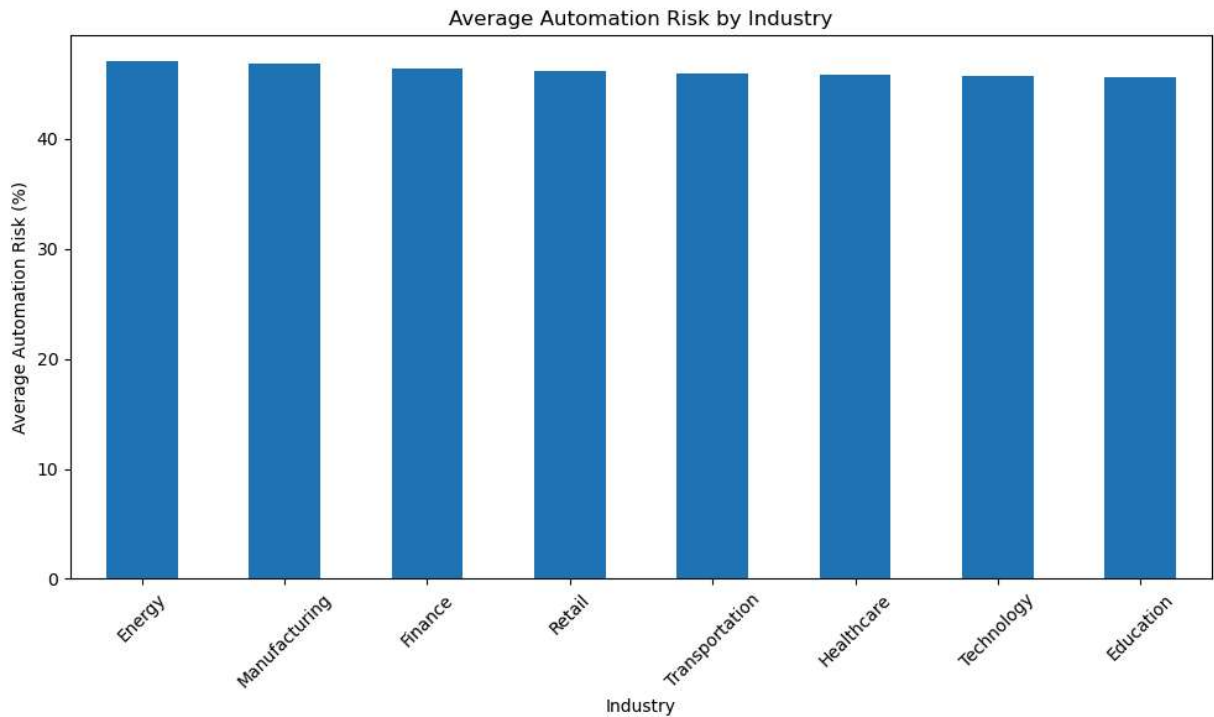
print("Cleaned dataset shape:", df_clean.shape)
```

Cleaned dataset shape: (15000, 20)

In [21]: *# 9. Bar Chart - Average Automation Risk by Industry*

```
industry_risk = df_clean.groupby("industry")["automation_risk_percent"].mean().sort
```

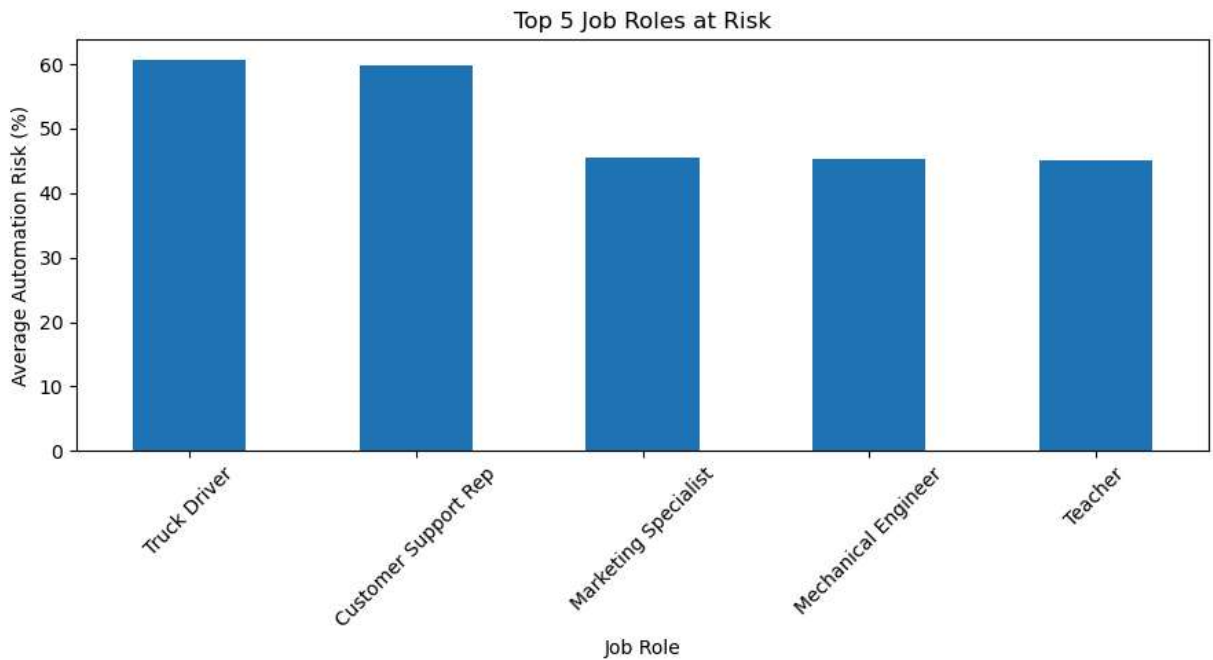
```
plt.figure(figsize=(10, 6))
industry_risk.plot(kind="bar")
plt.title("Average Automation Risk by Industry")
plt.xlabel("Industry")
plt.ylabel("Average Automation Risk (%)")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



In [23]: # 10. Top 5 Job Roles at Risk

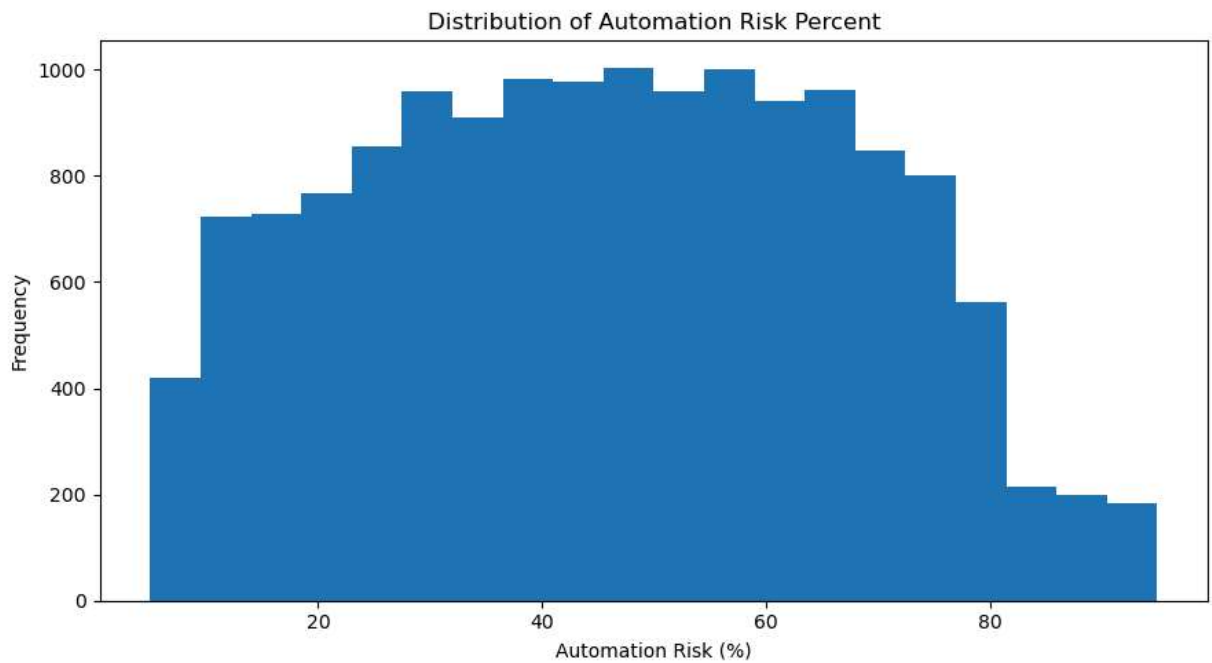
```
top_roles = df_clean.groupby("job_role")["automation_risk_percent"].mean().sort_val

plt.figure(figsize=(9, 5))
top_roles.plot(kind="bar")
plt.title("Top 5 Job Roles at Risk")
plt.xlabel("Job Role")
plt.ylabel("Average Automation Risk (%)")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



In [25]: # 11. Histogram - Automation Risk Distribution

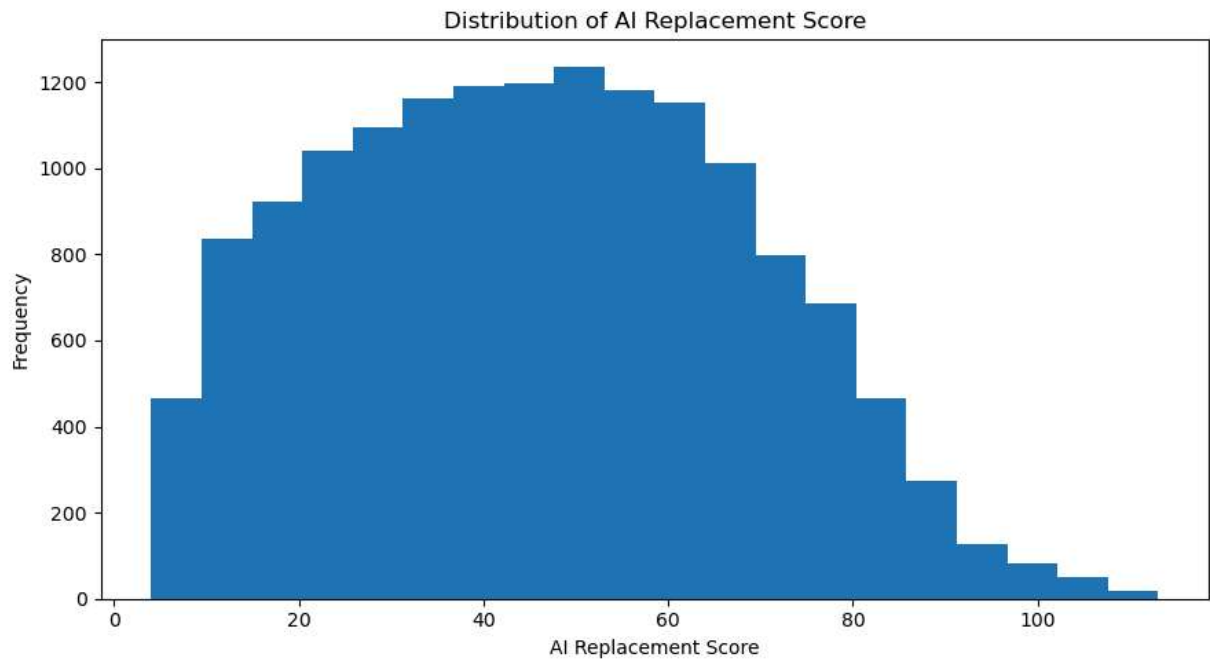
```
plt.figure(figsize=(9, 5))
plt.hist(df_clean["automation_risk_percent"], bins=20)
plt.title("Distribution of Automation Risk Percent")
plt.xlabel("Automation Risk (%)")
plt.ylabel("Frequency")
plt.tight_layout()
plt.show()
```



In [27]: # 12. Histogram - AI Replacement Score Distribution

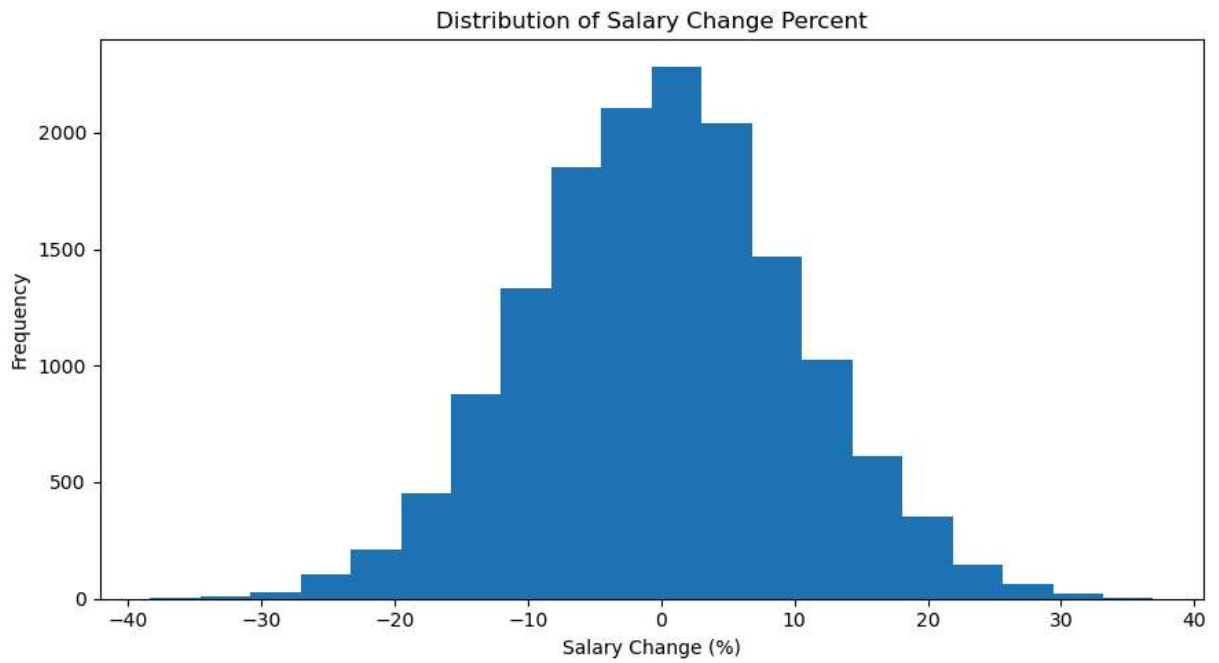
```
plt.figure(figsize=(9, 5))
plt.hist(df_clean["ai_replacement_score"], bins=20)
```

```
plt.title("Distribution of AI Replacement Score")
plt.xlabel("AI Replacement Score")
plt.ylabel("Frequency")
plt.tight_layout()
plt.show()
```



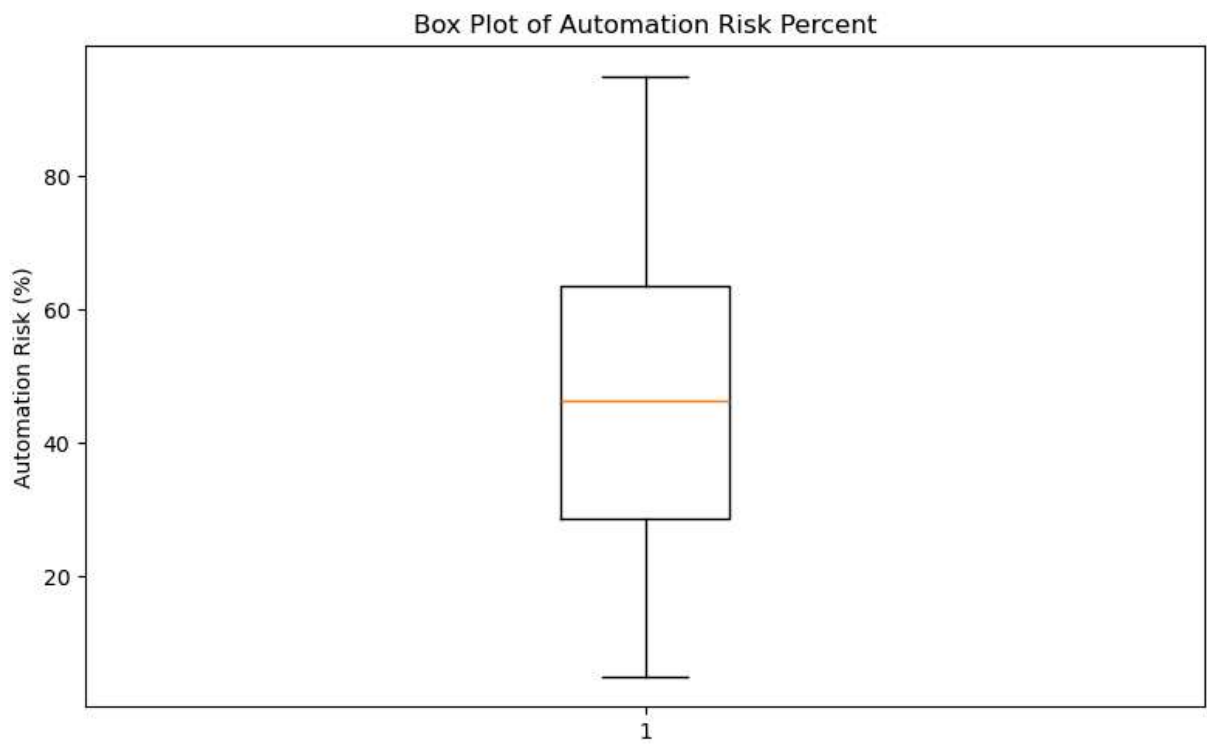
In [29]: *# 13. Histogram - Salary Change Percent Distribution*

```
plt.figure(figsize=(9, 5))
plt.hist(df_clean["salary_change_percent"], bins=20)
plt.title("Distribution of Salary Change Percent")
plt.xlabel("Salary Change (%)")
plt.ylabel("Frequency")
plt.tight_layout()
plt.show()
```



In [31]: *# 14. Box Plot - Check outliers in Automation Risk*

```
plt.figure(figsize=(8, 5))
plt.boxplot(df_clean["automation_risk_percent"])
plt.title("Box Plot of Automation Risk Percent")
plt.ylabel("Automation Risk (%)")
plt.tight_layout()
plt.show()
```



In [33]: *# 15. Remove outliers using IQR method*

```
Q1 = df_clean["automation_risk_percent"].quantile(0.25)
```



```

Q3 = df_clean["automation_risk_percent"].quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

df_no_outliers = df_clean[
    (df_clean["automation_risk_percent"] >= lower_bound) &
    (df_clean["automation_risk_percent"] <= upper_bound)
]

print("Original rows:", df_clean.shape[0])
print("Rows after removing outliers:", df_no_outliers.shape[0])

```

Original rows: 15000

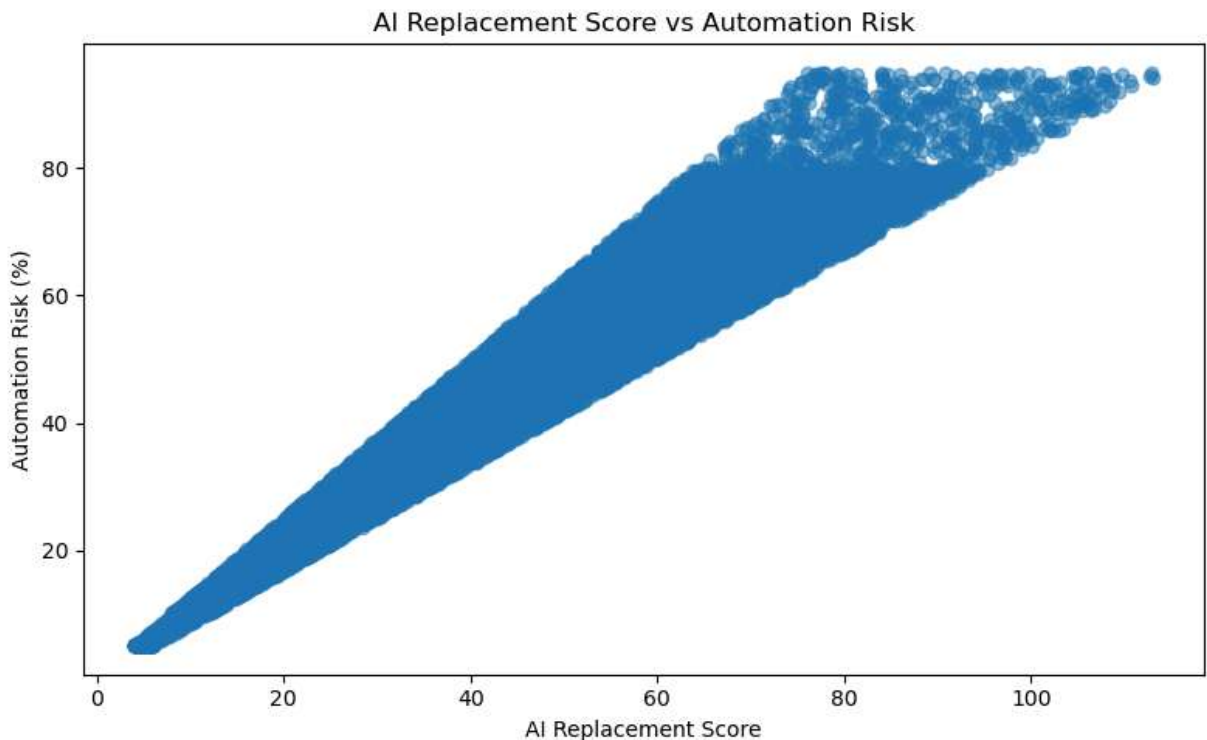
Rows after removing outliers: 15000

In [35]: # 16. Scatter Plot - AI Replacement Score vs Automation Risk

```

plt.figure(figsize=(8, 5))
plt.scatter(df_no_outliers["ai_replacement_score"], df_no_outliers["automation_risk_percent"])
plt.title("AI Replacement Score vs Automation Risk")
plt.xlabel("AI Replacement Score")
plt.ylabel("Automation Risk (%)")
plt.tight_layout()
plt.show()

```



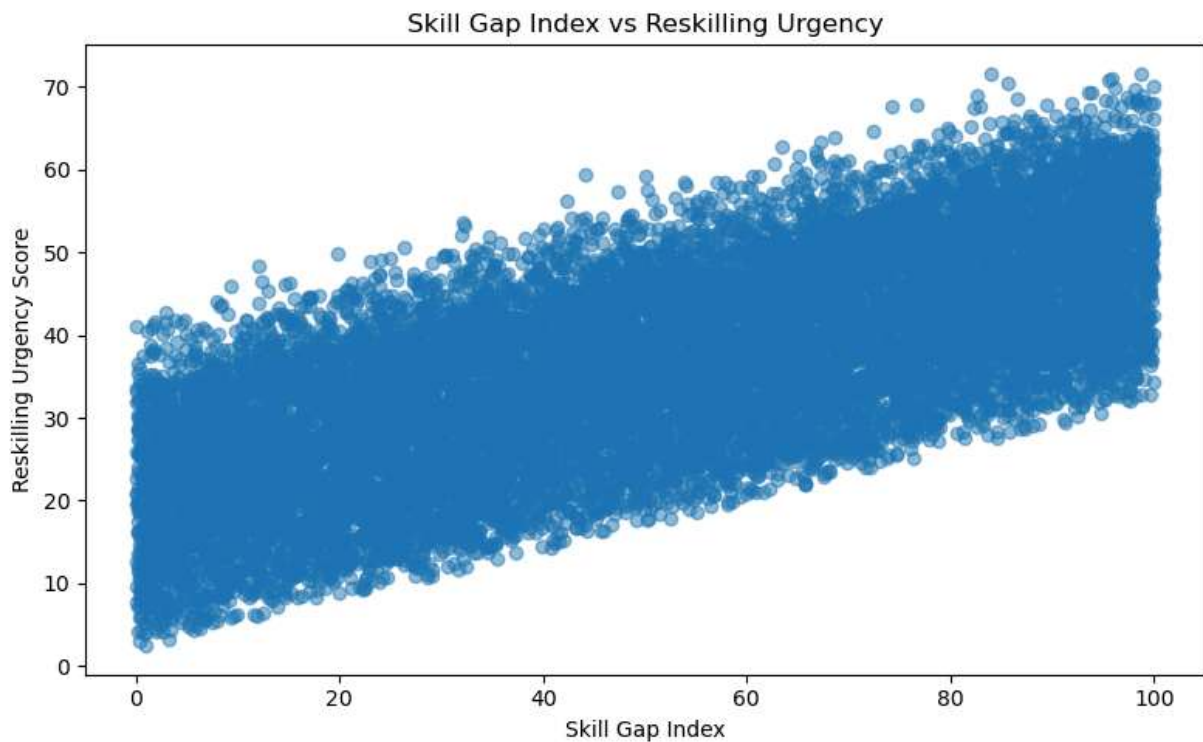
In [37]: # 17. Scatter Plot - Skill Gap Index vs Reskilling Urgency

```

plt.figure(figsize=(8, 5))
plt.scatter(df_no_outliers["skill_gap_index"], df_no_outliers["reskilling_urgency_score"])
plt.title("Skill Gap Index vs Reskilling Urgency")
plt.xlabel("Skill Gap Index")

```

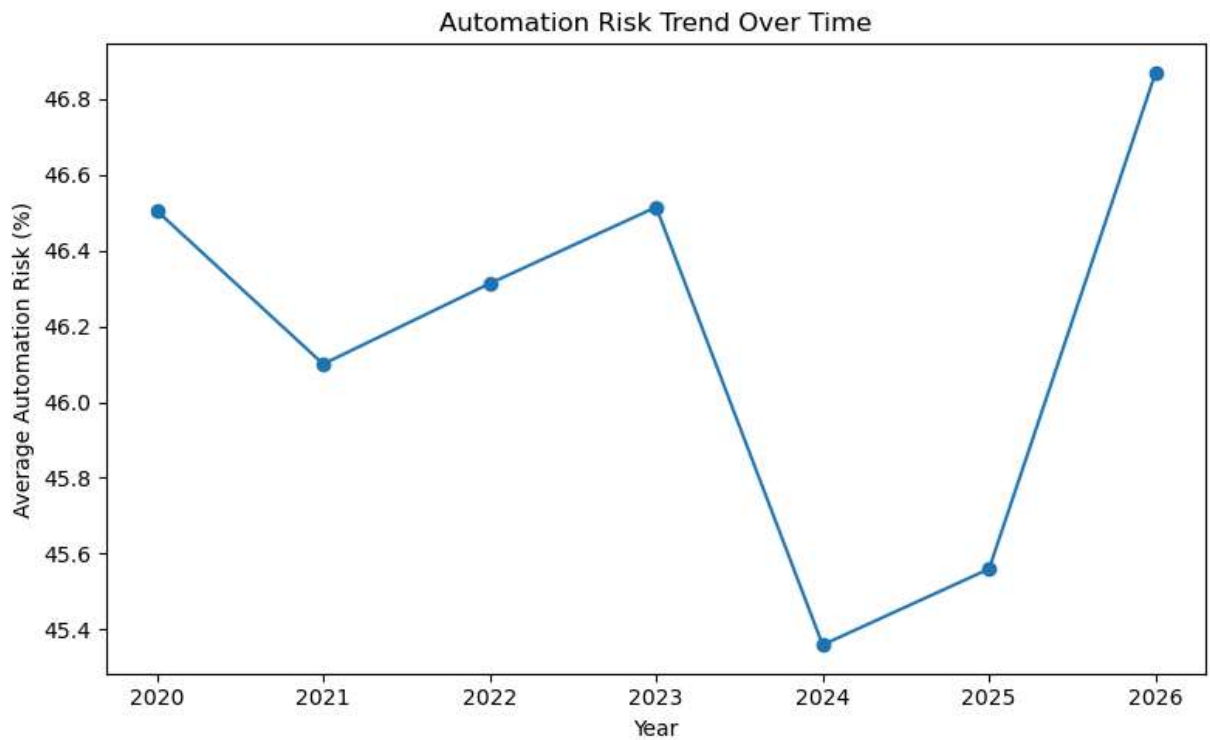
```
plt.ylabel("Reskilling Urgency Score")
plt.tight_layout()
plt.show()
```



In [39]: *# 18. Line Chart - Automation Risk Trend by Year*

```
yearly_risk = df_no_outliers.groupby("year")["automation_risk_percent"].mean()

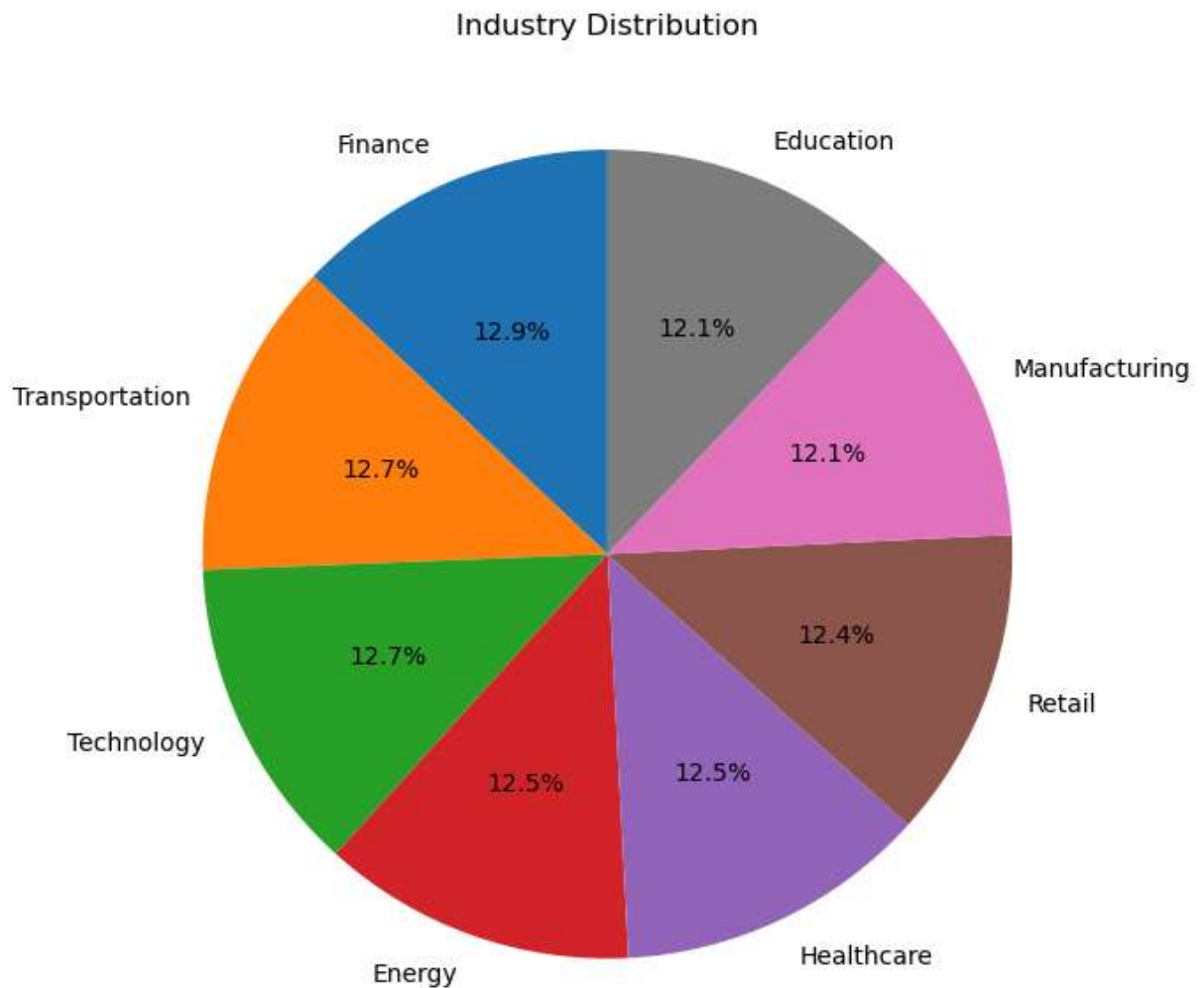
plt.figure(figsize=(8, 5))
plt.plot(yearly_risk.index, yearly_risk.values, marker="o")
plt.title("Automation Risk Trend Over Time")
plt.xlabel("Year")
plt.ylabel("Average Automation Risk (%)")
plt.xticks(yearly_risk.index)
plt.tight_layout()
plt.show()
```



```
In [41]: # 19. Pie Chart - Industry Distribution

industry_counts = df_no_outliers["industry"].value_counts()

plt.figure(figsize=(7, 7))
plt.pie(industry_counts, labels=industry_counts.index, autopct="%1.1f%", startangle=1
plt.title("Industry Distribution")
plt.tight_layout()
plt.show()
```

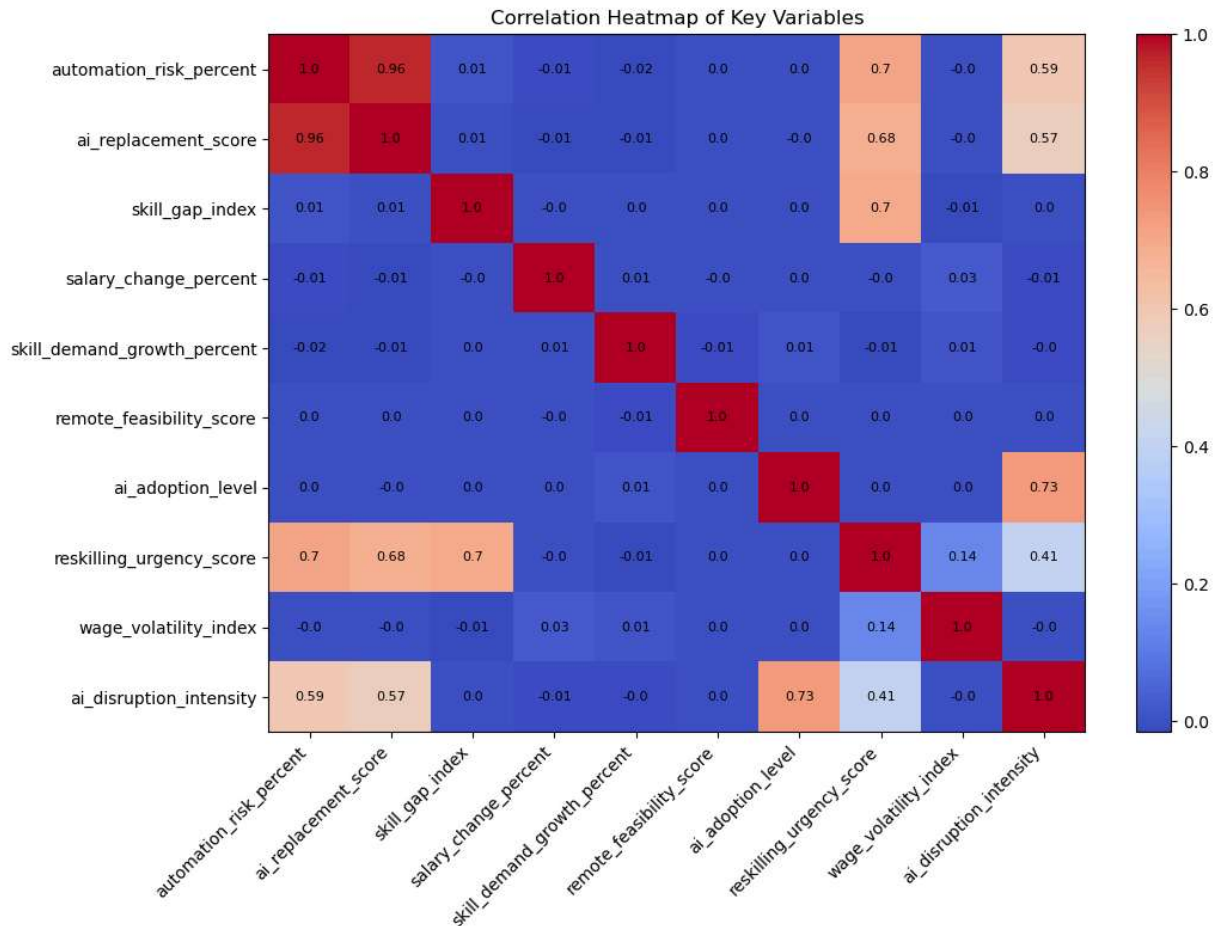


In [43]: *# 20. Correlation Heatmap - Important numeric variables only*

```
heatmap_cols = [  
    "automation_risk_percent",  
    "ai_replacement_score",  
    "skill_gap_index",  
    "salary_change_percent",  
    "skill_demand_growth_percent",  
    "remote_feasibility_score",  
    "ai_adoption_level",  
    "reskilling_urgency_score",  
    "wage_volatility_index",  
    "ai_disruption_intensity"  
]  
  
corr = df_no_outliers[heatmap_cols].corr()  
  
plt.figure(figsize=(11, 8))  
plt.imshow(corr, cmap="coolwarm", aspect="auto")  
plt.colorbar()  
  
plt.xticks(range(len(corr.columns)), corr.columns, rotation=45, ha="right")  
plt.yticks(range(len(corr.columns)), corr.columns)
```

```
# Add values inside heatmap
for i in range(len(corr.columns)):
    for j in range(len(corr.columns)):
        plt.text(j, i, round(corr.iloc[i, j], 2), ha="center", va="center", fontsize=10)

plt.title("Correlation Heatmap of Key Variables")
plt.tight_layout()
plt.show()
```



In [45]: # 21. Find strongest correlations with Automation Risk

```
target_corr = corr["automation_risk_percent"].sort_values(ascending=False)
target_corr
```

```
Out[45]: automation_risk_percent      1.000000
ai_replacement_score      0.964407
reskilling_urgency_score   0.704172
ai_disruption_intensity    0.590252
skill_gap_index           0.009731
ai_adoption_level         0.001905
remote_feasibility_score   0.001511
wage_volatility_index     -0.003448
salary_change_percent     -0.005281
skill_demand_growth_percent -0.015765
Name: automation_risk_percent, dtype: float64
```

```

In [47]: # 22. Feature Importance using Random Forest

# Target variable
y = df_no_outliers["automation_risk_percent"]

# Select useful features
features = [
    "ai_replacement_score",
    "skill_gap_index",
    "salary_before_usd",
    "salary_after_usd",
    "salary_change_percent",
    "skill_demand_growth_percent",
    "remote_feasibility_score",
    "ai_adoption_level",
    "education_requirement_level",
    "skill_transition_pressure",
    "wage_volatility_index",
    "reskilling_urgency_score",
    "ai_disruption_intensity",
    "industry",
    "job_role",
    "country"
]

X = df_no_outliers[features]

# Convert categorical columns into dummy variables
X = pd.get_dummies(X, drop_first=True)

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Model
rf_model = RandomForestRegressor(random_state=42)
rf_model.fit(X_train, y_train)

# Prediction
y_pred = rf_model.predict(X_test)

print("MAE:", mean_absolute_error(y_test, y_pred))
print("R2 Score:", r2_score(y_test, y_pred))

```

MAE: 1.8629132333333334
R2 Score: 0.9868854264218628

```

In [48]: # 23. Top 10 Important Features

feature_importance = pd.DataFrame({
    "Feature": X.columns,
    "Importance": rf_model.feature_importances_
}).sort_values(by="Importance", ascending=False).head(10)

```

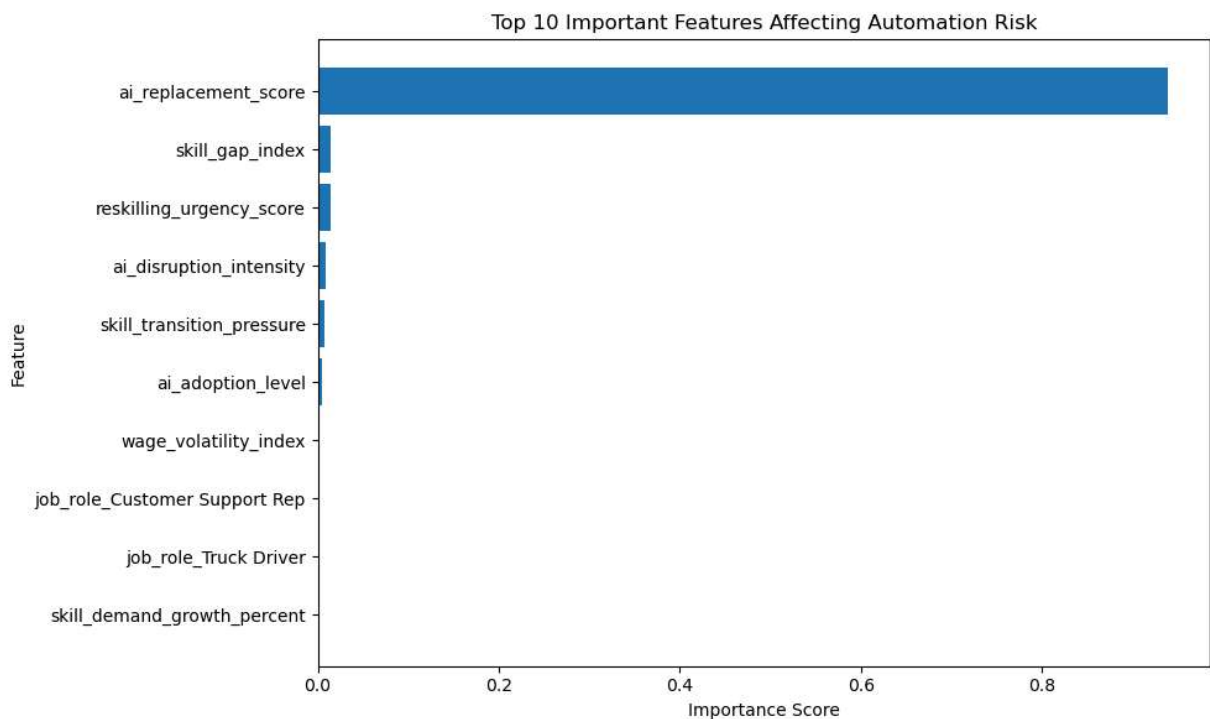
```
feature_importance
```

Out[48]:

	Feature	Importance
0	ai_replacement_score	0.938773
1	skill_gap_index	0.014243
11	reskilling_urgency_score	0.013468
12	ai_disruption_intensity	0.008588
9	skill_transition_pressure	0.007827
7	ai_adoption_level	0.004356
10	wage_volatility_index	0.001916
20	job_role_Customer Support Rep	0.001727
28	job_role_Truck Driver	0.001669
5	skill_demand_growth_percent	0.001135

In [49]: *# 24. Plot Top 10 Important Features*

```
plt.figure(figsize=(10, 6))
plt.barh(feature_importance["Feature"], feature_importance["Importance"])
plt.title("Top 10 Important Features Affecting Automation Risk")
plt.xlabel("Importance Score")
plt.ylabel("Feature")
plt.gca().invert_yaxis()
plt.tight_layout()
plt.show()
```



In [51]: # 25. Final Dashboard-Style Summary KPIs

```
avg_risk = df_clean["automation_risk_percent"].mean()
max_risk = df_clean["automation_risk_percent"].max()
total_jobs = df_clean.shape[0]

print("Average Automation Risk:", round(avg_risk, 2), "%")
print("Highest Automation Risk:", round(max_risk, 2), "%")
print("Total Job Roles Analyzed:", total_jobs)
```

Average Automation Risk: 46.18 %

Highest Automation Risk: 94.98 %

Total Job Roles Analyzed: 15000

In [90]: **import** matplotlib.pyplot **as** plt
import numpy **as** np
import textwrap

```
# ===== KPI CALCULATIONS =====
avg_risk = df_clean["automation_risk_percent"].mean()
max_risk = df_clean["automation_risk_percent"].max()
total_jobs = df_clean.shape[0]

# ===== FIGURE LAYOUT =====
fig = plt.figure(figsize=(20, 12))
fig.patch.set_facecolor('#EEF3F8') # ★ NEW BACKGROUND COLOR

gs = fig.add_gridspec(3, 3, height_ratios=[0.3, 2, 2])

# ===== TITLE =====
fig.suptitle("AI Job Automation Risk Dashboard",
             fontsize=26, fontweight="bold", y=0.965)

# ===== KPI SECTION =====
kpi = fig.add_subplot(gs[0, :])
kpi.axis("off")

fig.text(0.2, 0.90, f"{avg_risk:.2f}%", fontsize=22, fontweight="bold", ha="center")
fig.text(0.2, 0.87, "Average Automation Risk", fontsize=11, ha="center")

fig.text(0.5, 0.90, f"{max_risk:.2f}%", fontsize=22, fontweight="bold", ha="center")
fig.text(0.5, 0.87, "Highest Automation Risk", fontsize=11, ha="center")

fig.text(0.8, 0.90, f"{total_jobs:,}", fontsize=22, fontweight="bold", ha="center")
fig.text(0.8, 0.87, "Total Job Roles", fontsize=11, ha="center")

# ===== CHART GRID =====
axes = [fig.add_subplot(gs[i, j]) for i in range(1, 3) for j in range(3)]

# ★ KEEP CHARTS WHITE (clean look)
for ax in axes:
    ax.set_facecolor('#FFFFFF')

# ===== 1. Histogram =====
axes[0].hist(df_clean["automation_risk_percent"], bins=20)
axes[0].set_title("Distribution of Automation Risk")
```



```

axes[0].set_xlabel("Automation Risk (%)")
axes[0].set_ylabel("Frequency")

# ===== 2. Top 5 Job Roles =====
top_roles = df_clean.groupby("job_role")["automation_risk_percent"].mean().sort_val
wrapped_roles = [textwrap.fill(role, width=12) for role in top_roles.index]

axes[1].barh(wrapped_roles, top_roles.values)
axes[1].set_title("Most Vulnerable Job Roles to Automation")
axes[1].set_xlabel("Avg Automation Risk (%)")
axes[1].set_ylabel("Job Role")
axes[1].invert_yaxis()

# ===== 3. Trend =====
yearly_risk = df_clean.groupby("year")["automation_risk_percent"].mean()
axes[2].plot(yearly_risk.index, yearly_risk.values, marker="o")
axes[2].set_title("Automation Risk Trend Over Time")
axes[2].set_xlabel("Year")
axes[2].set_ylabel("Avg Automation Risk (%)")

# ===== 4. PIE =====
industry_counts = df_clean["industry"].value_counts()
axes[3].pie(industry_counts, labels=industry_counts.index, autopct="%1.1f%%", start
axes[3].set_title("Industry Distribution")

# ===== 5. SCATTER =====
axes[4].scatter(df_clean["ai_replacement_score"], df_clean["automation_risk_percent
axes[4].set_title("AI Replacement Score vs Automation Risk")
axes[4].set_xlabel("AI Replacement Score")
axes[4].set_ylabel("Automation Risk (%)")

# ===== 6. HEATMAP =====
cols = [
    "automation_risk_percent",
    "ai_replacement_score",
    "skill_gap_index",
    "salary_change_percent",
    "remote_feasibility_score",
    "reskilling_urgency_score"
]

corr = df_clean[cols].corr()
im = axes[5].imshow(corr, cmap="coolwarm", aspect="auto")

axes[5].set_title("Correlation Heatmap")
axes[5].set_xticks(range(len(cols)))
axes[5].set_yticks(range(len(cols)))
axes[5].set_xticklabels(cols, rotation=45, ha="right")

axes[5].set_yticklabels([])
axes[5].set_ylabel("")
axes[5].set_xlabel("Features")

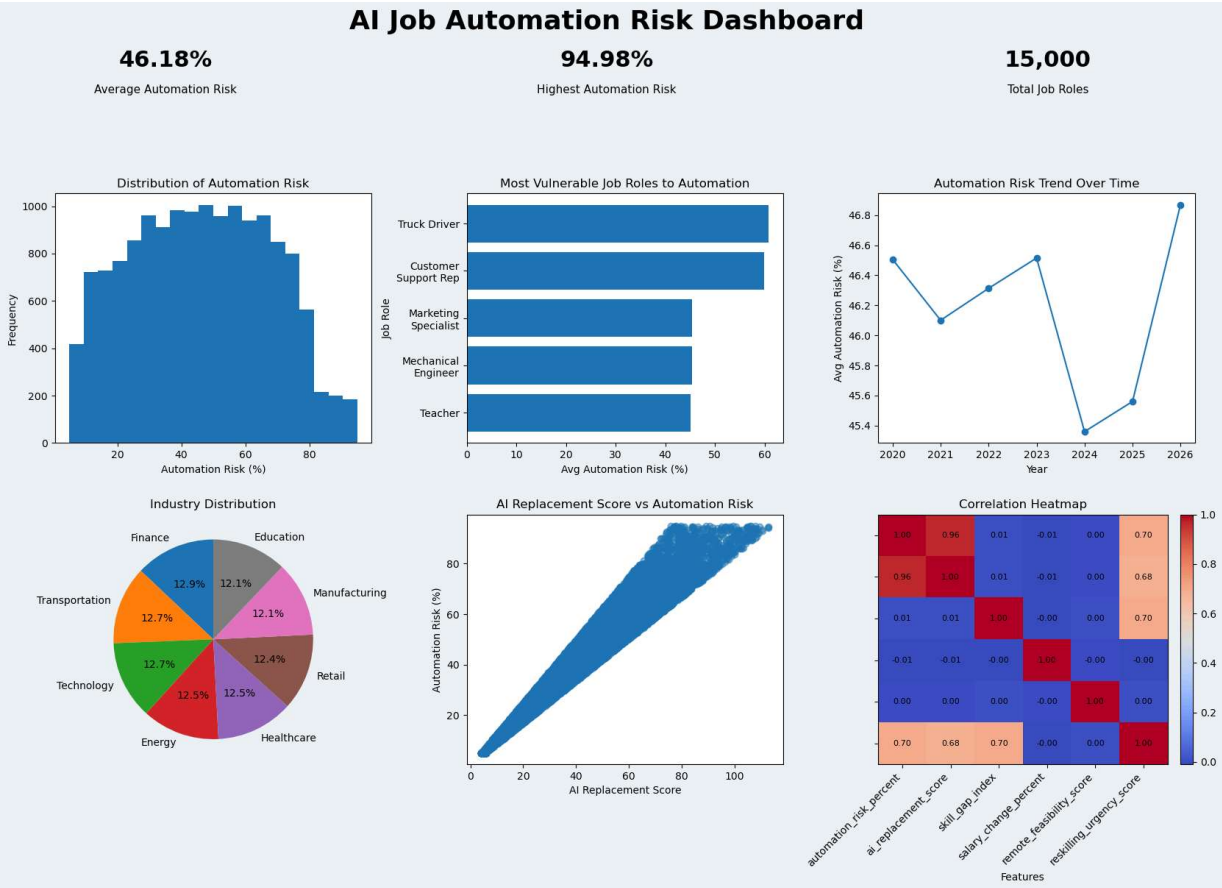
for i in range(len(cols)):
    for j in range(len(cols)):
        axes[5].text(j, i, f"{corr.iloc[i, j]:.2f}", ha="center", va="center", font

```

```
fig.colorbar(im, ax=axes[5], fraction=0.046, pad=0.04)

# ===== FINAL SPACING =====
plt.subplots_adjust(top=0.88, hspace=0.40, wspace=0.30)

plt.show()
```



In []: